# Interview Q&A Prep (10+ Yrs): Category-wise Answers

**Rule:** pehle 1-2 line ka direct answer, phir detail. Har answer mein apne project ka 1 real example jodo ("in my project..."), warna answer theory lagta hai.

**Categories:** 1 Microfrontend + Integration · 2 React + App Design · 3 HTML + Browser Storage + Login · 4 Security + SSO · 5 Performance · 6 Delivery + Testing · 7 Leadership + Behavioral

## Category 1: Microfrontend + Frontend-Backend Integration

### What is microfrontend architecture?

**Bolo:** "Microfrontend splits a large frontend into smaller, independently developed and deployed apps, each owned by a team, composed together in a shell at runtime. It's the microservices idea applied to UI."

- **Benefits:** independent deploys, team autonomy, tech flexibility, smaller codebases
- **Costs:** shared dependency/versioning, consistent UX, performance overhead, more infra
- **Memory trick: "Alag-alag dukaan, ek mall."** Har dukaan (MFE) independent, mall (shell) sabko jodta hai.

### How do you integrate different applications in a microfrontend?

**Bolo:** "A host/shell app loads remotes at runtime. I'd use Webpack Module Federation, with the shell handling routing and shared auth, and remotes exposing components or routes."

- **Options:** Module Federation (most common), single-spa, iframes (strong isolation, poor UX), Web Components, build-time npm packages (tight coupling)
- **Shared deps:** react, react-dom as singletons to avoid duplicate bundles
- **Communication:** URL/route params, custom events, shared event bus or small shared store. Avoid tight coupling
- **Auth:** shell owns login/token; remotes consume it
- **Design system:** shared component library for consistent UI

### How do you integrate the frontend with the backend?

**Bolo:** "Through REST or GraphQL APIs over HTTPS, using a centralized API layer with auth headers, error handling and typed contracts."

- Central API client (axios/fetch) with **interceptors**: attach token, handle 401 (refresh/redirect), global error handling
- Server state via React Query/RTK Query (caching, retries, loading/error states)
- Env-based base URLs; CORS configured on server or proxy in dev
- Contract via **OpenAPI/Swagger** so types are generated
- Large systems: **BFF (Backend for Frontend)** to aggregate calls

### What will you do if the backend does not expose an API?

**Bolo:** "I'd first clarify the contract with the backend team. To avoid being blocked, I'd work against a mock, and if needed build a thin API/BFF layer, but never connect the frontend directly to the database."

1. Agree a **contract-first** API spec (OpenAPI)
2. Mock with **MSW / json-server** so frontend continues in parallel
3. If backend truly can't deliver, build a thin **Node BFF** exposing the needed endpoints
4. Raise it as a risk/dependency with the manager early

## Category 2: React + App Design

### What is React Router?

**Bolo:** "React Router is the standard library for client-side routing in React. It maps URLs to components without full page reloads."

- Core: `BrowserRouter`, `Routes`, `Route`, `Link/NavLink`, `useNavigate`, `useParams`, `useSearchParams`
- Nested routes + `Outlet`, **protected routes** (auth guard), lazy loading routes with `React.lazy` for code splitting

### How do you manage sessions and data in a React application?

**Bolo:** "I separate server state, client state and auth. Server data via React Query, UI/global state via Context or Redux/Zustand, and the session via a backend-set HttpOnly cookie."

- **Server state:** React Query (cache, refetch, invalidation)
- **Client state:** local `useState`, Context for small global, Redux/Zustand for complex
- **Session/auth:** HttpOnly + Secure + SameSite cookie, tokens not in localStorage; on app load call `/me` to restore user
- **Memory trick: "S-C-A"** = Server state, Client state, Auth

## How do you persist data across multiple steps in an onboarding process?

**Bolo:** "I keep step data in a shared store, mirror it to the backend as a draft per step, and use a route per step, so refresh or resume on another device doesn't lose data."

- Shared state (Context/Redux/Zustand or form library like React Hook Form across steps)
- **Best:** save draft to backend after each step (PATCH). Works across devices
- **Light option:** sessionStorage for non-sensitive draft; never store sensitive data
- Step URL (`/onboarding/step-2`) + guard so user can't skip required steps
- Validate per step; clear draft after final submit

## How would you develop a To-Do application for managers and employees?

**Bolo:** "Role-based app: backend enforces RBAC, frontend shows UI by role. Managers create and assign tasks and see their team's; employees see and update their own."

- **Model:** Task {id, title, description, assigneeId, createdBy, status, dueDate, version}
- **Roles** come from token claims; UI hides actions, but **backend is the real enforcement**
- **APIs:** `GET /tasks` (filtered by role), `POST /tasks` (manager), `PATCH /tasks/:id`
- **Frontend:** React Query for list, filters, optimistic updates, route guards

## How would you allow managers and employees to see and update the same tasks?

**Bolo:** "One source of truth in the backend. Both roles read the same task, with field-level permissions on updates, and concurrency handled with versioning plus real-time or polling refresh."

- **Permissions:** employee updates status/comments only; manager edits everything
- **Conflicts:** optimistic locking (`version` / ETag); on mismatch, show "task changed, reload"
- **Freshness:** WebSocket/SSE for live updates, or polling / refetch on focus
- Audit trail: who changed what and when

# Category 3: HTML + Browser Storage + Login

## What is semantic HTML?

**Bolo:** "Using HTML elements that describe their meaning, like header, nav, main, article, section, footer, button, instead of generic divs."

- **Why:** accessibility (screen readers), SEO, readability, maintainability
- Example: `<button>` gives keyboard + role for free; `<div onClick>` doesn't

## What are the different types of browser storage?

**Memory trick: "C-L-S-I-C"** = Cookies, LocalStorage, SessionStorage, IndexedDB, Cache API.

| Storage | Size | Lifetime | Sent to server? | Notes |
|---|---|---|---|---|
| Cookies | ~4 KB | Session or expiry date | Yes, every request | HttpOnly protects from JS/XSS |
| localStorage | ~5 MB | Until cleared | No | Sync, readable by JS, XSS risk |
| sessionStorage | ~5 MB | Per tab, till tab closes | No | Good for temp UI state |
| IndexedDB | Large | Persistent | No | Async, structured data, offline |
| Cache API / Service Worker | Large | Persistent | No | Offline assets/PWA |

## How can a user remain logged in after closing and reopening the browser?

**Bolo:** "Use a persistent session: a cookie with an expiry (not a session cookie), backed by a server-side session or refresh token."

- Cookie with `Max-Age/Expires`, flags **HttpOnly, Secure, SameSite**
- Backend session (e.g., Redis) or **refresh token with rotation**; short-lived access token
- On app load, call `/me` or refresh endpoint to restore state
- With SSO: IdP session still alive, so silent re-auth works

- "Remember me" checkbox decides persistent vs session cookie

# Category 4: Security + SSO

## What is SSO?

**Bolo:** "SSO lets a user log in once with a central Identity Provider and access many apps without logging in again. Apps never see the password; they trust a signed token from the IdP, via SAML 2.0 or OpenID Connect."

**Memory trick: Gate-Band-Ride.** Gate = IdP, Wristband = Token, Ride = App.

## How does SSO work across multiple applications?

**Flow: R-L-T-V-S**

1. **R**edirect to IdP
2. **L**ogin at IdP (+ MFA)
3. **T**oken issued (signed assertion / ID token)
4. **V**erify signature, `iss`, `aud`, `exp`, nonce
5. **S**ession created in the app

**Second app:** user redirected to IdP, IdP session cookie already exists, so token issued with no login page. That's the SSO.

**Tricky points:** two independent sessions (IdP vs app), so IdP logout doesn't auto-logout apps (fix: back-channel logout / short sessions). SSO is the goal; SAML/OIDC are protocols.

## How do you secure a web application at runtime?

**Bolo:** "Defense in depth: secure transport, strong auth/authz on the server, protection against XSS/CSRF/injection, secure headers, least privilege, and monitoring."

- **HTTPS + HSTS**
- **XSS:** output encoding, sanitize HTML, strict **CSP**, avoid `dangerouslySetInnerHTML`
- **CSRF:** SameSite cookies + CSRF tokens
- **Auth:** HttpOnly Secure cookies, MFA, short token expiry
- **Server-side** validation and authorization on every request

- **Headers:** CSP, X-Content-Type-Options, X-Frame-Options/frame-ancestors, Referrer-Policy
- Rate limiting, WAF, dependency scanning (`npm audit`), no secrets in frontend, logging + alerts

### How do you handle security in a multi-tenant application?

**Bolo:** "The core rule is strict tenant isolation: tenant identity comes from the verified token, never from client input, and every data access is scoped by tenant."

- **Tenant ID from token/session**, not request body or URL
- Tenant filter on **every query** (or row-level security / separate schema/DB for high isolation)
- **Per-tenant IdP config**; token `iss` must match that tenant's issuer
- Per-tenant roles/permissions; cache keys and queues include tenant ID
- Per-tenant rate limits, encryption keys (if required), audit logs
- Test for **cross-tenant leaks (IDOR)**

### What is the Same-Origin Policy?

**Bolo:** "A browser security rule that stops a script from one origin reading data from another origin. Origin = scheme + host + port."

- `https://a.com` vs `https://a.com:8443` vs `http://a.com` vs `https://api.a.com` = all different origins
- It restricts **reading** responses; it protects users from a malicious site reading their data from another site

### What is CORS?

**Bolo:** "CORS is the mechanism that lets a server explicitly relax the Same-Origin Policy for specific origins, using Access-Control headers."

- Server sends `Access-Control-Allow-Origin` (and methods, headers)
- **Preflight:** browser sends `OPTIONS` first for non-simple requests (custom headers, PUT/DELETE, JSON content type)
- With cookies: `Allow-Credentials: true` and origin must be explicit, **not** `*`
- CORS is **browser-enforced**; it doesn't protect the server from curl/Postman
- Dev fix: proxy. Prod fix: configure server correctly

# Category 5: Performance

## How would you optimize a news or content-heavy website?

**Bolo:** "Render fast, cache aggressively, and keep heavy stuff (images, ads, third-party scripts) from blocking content."

- **SSR / SSG / ISR** for fast first paint + SEO
- **CDN** + cache headers, stale-while-revalidate
- **Images:** WebP/AVIF, `srcset`, lazy-load below the fold, preload hero image
- **Ads/third-party:** load async/deferred, **reserve space** to avoid layout shift
- Fonts: `font-display: swap`, preload key fonts, subset
- Code splitting, critical CSS, virtualize long lists, paginate

## How do you improve website loading speed?

**Memory trick: "R-C-I-C"** = Reduce, Cache, Images, Code-split.

- **Reduce:** minify, tree-shake, compress (Brotli/gzip), remove unused JS/CSS, fewer third-party scripts
- **Cache:** CDN, HTTP cache headers, service worker
- **Images:** modern formats, lazy-load, right sizing
- **Code-split:** route-based lazy loading, `defer` / `async` scripts
- `preconnect` / `preload` for critical resources, HTTP/2 or 3

## How do you measure frontend performance?

**Bolo:** "Both lab and field: Lighthouse/DevTools for lab debugging, and Real User Monitoring for actual user data."

- **Lab:** Lighthouse, WebPageTest, Chrome DevTools Performance, bundle analyzer
- **Field (RUM):** `web-vitals` library, CrUX, Datadog/New Relic/GA
- **CI:** performance budgets to stop regressions
- Always compare p75 values, not just averages

## What are Core Web Vitals?

**Bolo:** "Google's user-centric metrics for loading, interactivity and visual stability: LCP, INP and CLS."

| Metric | Measures | Good |
|---|---|---|
| **LCP** Largest Contentful Paint | Loading | ⩽ 2.5 s |
| **INP** Interaction to Next Paint | Responsiveness (replaced FID in 2024) | ⩽ 200 ms |
| **CLS** Cumulative Layout Shift | Visual stability | ⩽ 0.1 |

Assessed at the 75th percentile of real users.

### How do you measure page load time?

**Bolo:** "With the Navigation Timing API for exact numbers, DevTools Network for debugging, and RUM for real users."

- `performance.getEntriesByType('navigation')[0]` gives `domContentLoadedEventEnd`, `loadEventEnd`, `responseStart` (TTFB), etc.
- Page load time ≈ `loadEventEnd - startTime`
- But "load" alone isn't user-centric; prefer LCP and other vitals

### Difference between `DOMContentLoaded` and `load`?

**Bolo:** "DOMContentLoaded fires when the HTML is parsed and deferred scripts are done, without waiting for images. `load` fires later, after all resources like images, stylesheets and iframes are loaded."

**Memory trick:** DOM = **ghar ka dhancha** ready (structure). Load = **furniture bhi aa gaya** (sab resources).

## Category 6: Delivery + Testing

### Explain the process from local development to QA deployment.

**Bolo:** "Feature branch, local checks, pull request, automated CI, code review, merge, then the pipeline builds and deploys to QA for testing."

1. Create feature branch from develop; code + local tests
2. Pre-push checks (lint, tests, build)
3. Raise **PR**; **CI** runs lint, unit tests, build, security scan
4. **Code review** + approval

5. Merge to develop; **CD** builds artifact (e.g., Docker) and deploys to **QA**
6. Smoke test, then QA regression; fix bugs through the same flow
7. Promote to UAT / Prod after sign-off

### What checks do you perform before pushing code?

- Lint + format + type check (TypeScript)
- Unit tests pass; build succeeds
- Self-review the diff: no `console.log`, commented code, secrets
- Manually test the feature + edge cases
- Pull/rebase latest, resolve conflicts
- **Husky pre-commit hooks** to automate lint/test

### What testing phases do you perform before merging code?

**Memory trick: "U-I-E-R"** = Unit, Integration, E2E, Review.

- **Unit:** Jest + React Testing Library
- **Integration/component:** components with API mocks
- **E2E:** Cypress/Playwright on critical flows
- **Review:** peer review, accessibility check, CI green, coverage threshold

## Category 7: Leadership + Behavioral

### Responsibilities of a Senior Engineer / Tech Lead?

**Bolo:** "Own technical direction and delivery: design decisions, code quality, mentoring, and clear communication with stakeholders."

- Architecture and technical design, trade-off decisions
- Code reviews, standards, quality
- Mentoring, unblocking the team
- Estimation, planning, risk management
- Communicating status and risks to stakeholders
- Accountability for delivery, not just own tasks

## How do you handle project delays?

**Structure: Detect, Tell, Re-plan, Prevent**

1. **Detect** early and find the root cause (scope, dependency, estimate, blocker)
2. **Tell** stakeholders early with facts, not surprises
3. **Re-plan:** reprioritize, cut/phase scope (MVP first), add help, remove blockers
4. **Prevent:** retrospective, better estimates, risk tracking

## How do you manage stakeholder pressure?

**Bolo:** "By being transparent and giving options, not just saying yes or no."

- Understand the real business need behind the deadline
- Show data: progress, risks, dependencies
- Offer trade-offs: scope vs time vs resources
- Say no with an alternative; regular short updates
- Protect the team from noise

## How do you handle disagreements with a senior colleague?

**Bolo:** "I listen first to understand their reasoning, then present my view with data, privately and respectfully. If still unresolved, I suggest a POC or escalate the decision, and then disagree and commit."

- Focus on the problem, not the person
- Use facts, benchmarks, trade-offs
- Once decided, support it fully

## Where do you see yourself in the next few years?

**Bolo:** "I want to grow into a technical lead or architect role: owning larger system design, mentoring engineers, and driving frontend strategy, while staying hands-on."

- Keep it aligned to the role you're interviewing for
- Show growth in impact, not just title

# Final Notes

- **Honest feedback:** In senior interviews, long vague answers are the biggest risk. Pehle 1 line, phir detail.
- Behavioral answers (Category 7) ke liye apne real project ka ek **STAR example** (Situation, Task, Action, Result) ready rakho. Generic answer weak lagta hai.
- Note: question wording voice dictation se reconstructed hai, to concept pakdo, exact words nahi.